

Phase 2 — Exercices pour aller plus loin

Six exercices avancés sur le RAG avant la Phase 3

Cours PyTutor2

Résumé

Ce document complète `phase2.tex` avec six exercices RAG avancés : impact du `chunk_size` sur la qualité, mode interactif CLI, prompt strict anti-hallucination, observabilité avec LangSmith, `MultiQueryRetriever` pour reformuler les questions, et filtrage par métadonnée. Chaque exercice suit la même structure que les principaux.

Table des matières

1	Impact du <code>chunk_size</code> sur la qualité du RAG	2
1.1	Solution	2
1.1.1	Architecture	2
1.1.2	Le code complet	2
1.1.3	Explications pas à pas	4
1.2	Vérification dans LangSmith	4
2	Boucle interactive : transformer le RAG en mini-CLI	5
2.1	Solution	5
2.1.1	Architecture	5
2.1.2	Le code complet	6
2.1.3	Explications pas à pas	6
2.2	Vérification dans LangSmith	7
3	Prompt strict pour refuser le hors-contexte	8
3.1	Solution	8
3.1.1	Architecture	8
3.1.2	Le code complet	8
3.1.3	Explications pas à pas	9
3.2	Vérification dans LangSmith	10
4	Vérifier et exploiter LangSmith	11
4.1	Solution	11
4.1.1	Architecture	11
4.1.2	Le code complet	11
4.1.3	Explications pas à pas	12
4.2	Vérification dans LangSmith	13
5	MultiQueryRetriever : reformuler la question	14
5.1	Solution	14
5.1.1	Architecture	14
5.1.2	Le code complet	14
5.1.3	Explications pas à pas	15
5.2	Vérification dans LangSmith	16

6 Filtrage par métadonnée	17
6.1 Solution	17
6.1.1 Architecture	17
6.1.2 Le code complet	17
6.1.3 Explications pas à pas	18
6.2 Vérification dans LangSmith	19

1 Impact du `chunk_size` sur la qualité du RAG

Exercice bonus 1. Mesurer expérimentalement l'effet du découpage

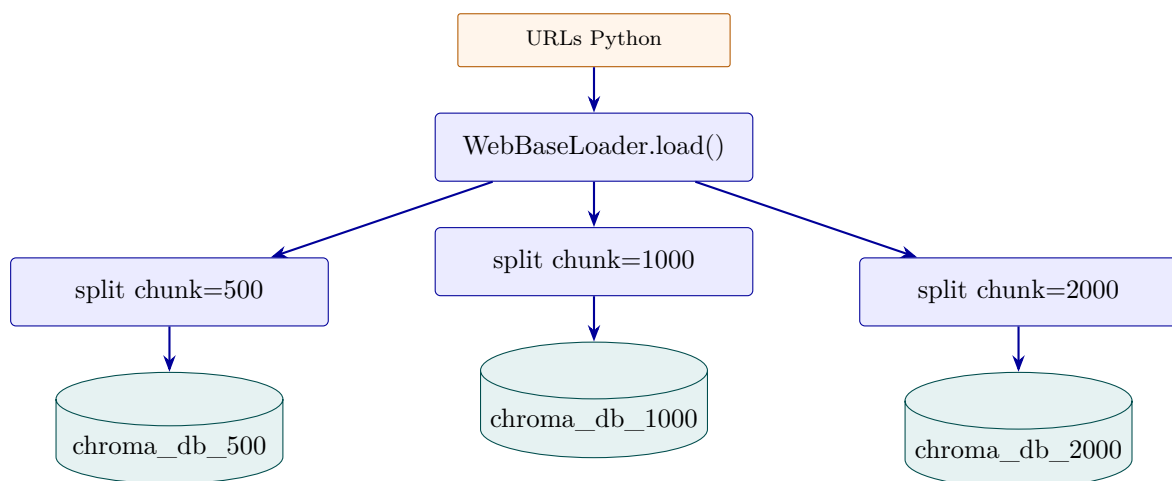
Construis trois vectorstores avec `chunk_size = 500, 1000, 2000` caractères (et `chunk_overlap` proportionnel, 20%). Pose la même question précise aux trois et compare (a) la longueur moyenne des chunks ramenés, (b) leur pertinence visuelle, (c) la qualité de la réponse finale. **Contrainte** : persister les trois index dans des dossiers séparés (`chroma_db_500`, etc.) pour éviter de réindexer à chaque test.

Objectifs pédagogiques

- Voir **concrètement** pourquoi le `chunk_size` est l'hyperparamètre n°1 de qualité RAG.
- Identifier le compromis *granularité vs contexte* : petit chunk = retrieval précis mais contexte fragmenté ; gros chunk = contexte riche mais retrieveur moins discriminant.
- Trouver expérimentalement le sweet spot pour un corpus donné.

1.1 Solution

1.1.1 Architecture



Trois index isolés, même corpus, paramètre de découpage différent. C'est le protocole expérimental le plus simple pour évaluer un hyperparamètre en RAG : *ceteris paribus*, on change une seule variable.

1.1.2 Le code complet

```

1 from pathlib import Path
2 from langchain_text_splitters import RecursiveCharacterTextSplitter
3 from langchain_chroma import Chroma
4 from langchain_community.document_loaders import WebBaseLoader
5
6 from phase2_rag import (

```

```

7     PYTHON_DOCS_URLS, COLLECTION_NAME, get_embeddings, build_rag_chain,
8 )
9
10
11 def build_vectorstore_with_chunk_size(chunk_size: int, persist_dir: str) -> Chroma:
12     """Construit (ou recharge) un vectorstore avec un chunk_size donné."""
13     embeddings = get_embeddings()
14     if Path(persist_dir).exists():
15         return Chroma(
16             collection_name=COLLECTION_NAME,
17             persist_directory=persist_dir,
18             embedding_function=embeddings,
19         )
20
21     loader = WebBaseLoader(PYTHON_DOCS_URLS)
22     loader.requests_kwargs = {"headers": {"User-Agent": "PyTutor-Edu/0.1"}}
23     raw_docs = loader.load()
24
25     splitter = RecursiveCharacterTextSplitter(
26         chunk_size=chunk_size,
27         chunk_overlap=int(chunk_size * 0.2),           # 20% proportionnel
28         separators=["\n\n", "\n", ". ", " ", ""],
29     )
30     chunks = splitter.split_documents(raw_docs)
31
32     return Chroma.from_documents(
33         documents=chunks, embedding=embeddings,
34         collection_name=COLLECTION_NAME, persist_directory=persist_dir,
35     )
36
37
38 def exercise_1() -> None:
39     configs = [
40         (500, "./chroma_db_500"),
41         (1000, "./chroma_db_1000"),
42         (2000, "./chroma_db_2000"),
43     ]
44     question = "Comment lève-t-on une exception personnalisée en Python ?"
45
46     for chunk_size, persist_dir in configs:
47         print(f"\n--- chunk_size = {chunk_size} ---")
48         vs = build_vectorstore_with_chunk_size(chunk_size, persist_dir)
49         retriever = vs.as_retriever(search_kwargs={"k": 3})
50
51         docs = retriever.invoke(question)
52         avg_len = sum(len(d.page_content) for d in docs) // len(docs)
53         print(f" Longueur moyenne des chunks : {avg_len} caractères")
54
55         chain = build_rag_chain(retriever)
56         answer = chain.invoke(question)
57         print(f" Réponse : {answer[:300]}...")

```

1.1.3 Explications pas à pas

Étape 1 — Le compromis granularité vs contexte

chunk_size	Avantages	Inconvénients
500	Recherche précise (chaque chunk est ciblé)	Contexte fragmenté, le modèle manque l'environnement de l'idée
1000 (défaut)	Bon compromis pour de la doc technique	—
2000	Contexte riche, chunks auto-suffisants	Beaucoup de bruit, le retriever distingue mal pertinent / tangentiel

Étape 2 — Mesurer plutôt que deviner

L'effet du `chunk_size` dépend du corpus. Sur la doc Python (passages bien structurés, paragraphes courts), 1000 est optimal. Sur des transcripts de conférences (idées qui s'étalent sur plusieurs paragraphes), 1500–2000 peut être meilleur. **Mesure** pour ton corpus.

*Le bon outil de mesure n'est pas la longueur ni le nombre de chunks : c'est la **qualité subjective des réponses** sur un échantillon de questions représentatives. Construis un mini-jeu de 10 questions typiques et compare les réponses.*

Étape 3 — Le `chunk_overlap` proportionnel

On garde `chunk_overlap = 0.2 × chunk_size` : 20% de chevauchement entre chunks consécutifs. C'est une bonne valeur *par défaut* qui évite de couper une idée en deux sans gonfler excessivement la taille de l'index.

Étape 4 — Persister chaque variante

Réindexer prend du temps. En persistant dans des dossiers séparés (`chroma_db_500`, etc.), on peut :

- Comparer les variantes rapidement (rechargement <1 s chacun).
- Garder un environnement reproductible.
- Faire le test une fois, l'oublier.

1.2 Vérification dans LangSmith

Pour chaque `chunk_size`, lance la même question et compare les runs côte-à-côte. Trois choses se révèlent :

1. **Tokens du prompt** : `chunk_size=500` avec `k=3` donne ~1500 tokens de contexte ; `chunk_size=2000` donne ~6000.
2. **Pertinence des chunks** : lisible dans l'output du retriever.
3. **Qualité de la réponse** : peut être annotée manuellement dans LangSmith (*thumbs up/down*) pour mémoriser l'évaluation.

Le coût en tokens monte vite avec le `chunk_size` : à `k=3`, passer de 1000 à 2000 caractères double le coût par requête sans nécessairement améliorer la qualité. Optimise.

Récapitulatif

Ce qu'il faut retenir :

1. Le `chunk_size` est l'hyperparamètre n°1 de qualité RAG. Mesure, ne devine pas.
2. Sweet spot général pour de la doc technique : **800–1200 caractères**, `overlap = 20%`.
3. Petit chunk : précis mais fragmenté. Gros chunk : riche mais bruité.
4. Persiste chaque variante dans son dossier pour comparer sans réindexer.
5. Le coût en tokens augmente linéairement avec `chunk_size × k`.

2 Boucle interactive : transformer le RAG en mini-CLI

Exercice bonus 2. Une boucle while pour explorer la doc librement

Transforme la chaîne RAG en une petite REPL (*Read-Eval-Print Loop*) : une boucle `while` qui demande une question à l'utilisateur via `input()`, appelle la chaîne, affiche la réponse **et les sources**, puis recommence. Termine proprement sur ligne vide ou `Ctrl+C`.

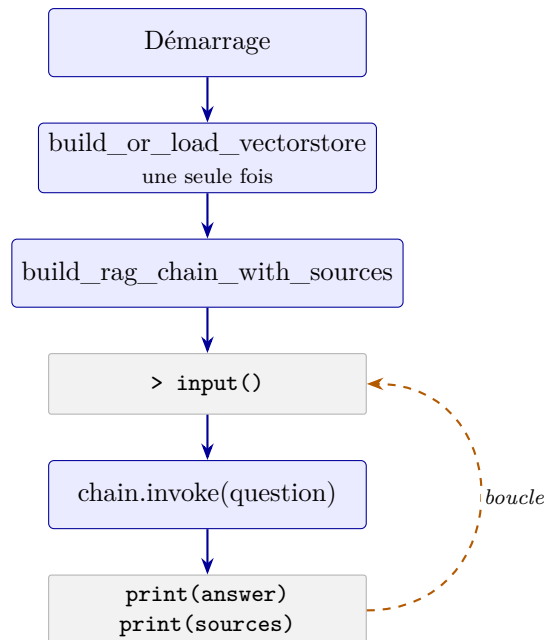
Contrainte : utiliser `build_rag_chain_with_sources` de l'exercice 4 pour afficher les sources, et gérer les exceptions `EOFError` / `KeyboardInterrupt` pour une sortie propre.

Objectifs pédagogiques

- Construire une mini-CLI qui sert de banc de test pour explorer son corpus librement.
- Gérer la terminaison propre (signal et ligne vide).
- Préparer mentalement la transition vers une UI web (Gradio, Streamlit) où la boucle d'événements est cachée par le framework.

2.1 Solution

2.1.1 Architecture



Trois éléments importants pour la performance perçue : charger le store **une seule fois** (lent), construire la chaîne **une seule fois**, et la réutiliser en boucle pour chaque question (rapide).

2.1.2 Le code complet

```
1 from phase2_rag import build_or_load_vectorstore, build_rag_chain_with_sources
2
3
4 def exercise_2_interactive() -> None:
5     print("PyTutor - mode interactif (Ctrl+C ou ligne vide pour quitter)")
6
7     # On charge le store et on construit la chaîne UNE SEULE FOIS
8     vectorstore = build_or_load_vectorstore()
9     retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
10    chain = build_rag_chain_with_sources(retriever)
11
12    while True:
13        try:
14            question = input("\n> ").strip()
15        except (EOFError, KeyboardInterrupt):
16            print("\nÀ la prochaine !")
17            break
18
19        if not question:
20            print("Au revoir.")
21            break
22
23        result = chain.invoke(question)
24
25        print("\n" + "-" * 70)
26        print(result["answer"])
27        print("-" * 70)
28        print("Sources :")
29        for i, doc in enumerate(result["context"], 1):
30            src = doc.metadata.get("source", "?").split("/)[-1]
31            print(f"  [{i}] {src}")
```

2.1.3 Explications pas à pas

Étape 1 — Init coûteux, hors de la boucle

```
1 vectorstore = build_or_load_vectorstore()      # 1s (rechargement)
2 retriever = vectorstore.as_retriever(...)      # instantané
3 chain = build_rag_chain_with_sources(...)      # instantané
4 # Boucle après l'init :
5 while True:
6     ...
```

Pattern universel des REPL : tout ce qui est lent (chargement, parsing, initialisation) se fait **une fois** avant la boucle. Dans la boucle, chaque tour ne fait que ce qui dépend de l'input.

Étape 2 — Gérer les sorties propres

Trois façons pour l'utilisateur de quitter :

- Ctrl+C : `input()` lève `KeyboardInterrupt`.
- Ctrl+D (EOF) : `input()` lève `EOFError`.
- Ligne vide : `input()` renvoie "", qu'on traite explicitement.

Le `try/except` attrape les deux premiers, le `if not question` attrape le troisième.

Sans `try/except`, un `Ctrl+C` propage une trace Python disgracieuse à l'utilisateur. L'attraper et imprimer un message amical (« À la prochaine ! ») est la marque du logiciel poli.

Étape 3 — Affichage avec sources

`build_rag_chain_with_sources` renvoie un dict avec `answer`, `context`, `question`. On affiche :

- La réponse, encadrée d'une ligne pour la séparer visuellement du prompt.
- Les sources, chacune avec son fichier (`.split("/")[-1]` pour ne garder que le *basename*).

Étape 4 — Vers une UI web

Cette mini-CLI est la base mentale d'une UI :

- **Gradio** :

```
1 import gradio as gr
2 gr.Interface(
3     fn=lambda q: chain.invoke(q)["answer"],
4     inputs="text", outputs="text",
5 ).launch()
```

- **Streamlit** : `st.text_input`, `st.write`.
- **FastAPI** : un endpoint `POST /ask` qui retourne le dict en JSON.

Dans tous les cas, la chaîne RAG reste un `Runnable` inchangé : on remplace juste l'interface utilisateur autour.

2.2 Vérification dans LangSmith

Chaque appel à `chain.invoke(...)` produit **une trace distincte** dans LangSmith. C'est un super outil pour :

- Repérer les questions qui ont mal marché.
- Comparer plusieurs reformulations d'une même question.
- Annoter manuellement les bonnes/mauvaises réponses (*thumbs up/down*) pour évaluer ton RAG.

Récapitulatif

Ce qu'il faut retenir :

1. Pattern *REPL* : *init* coûteux **hors** de la boucle, traitement rapide **dans** la boucle.
2. Gérer la terminaison propre : `EOFError`, `KeyboardInterrupt`, et ligne vide.
3. Une CLI interactive est le meilleur banc de test pour explorer son corpus et identifier les angles morts.
4. La chaîne RAG reste inchangée quand on passe à une UI web — c'est l'interface qui change, pas le *Runnable*.

3 Prompt strict pour refuser le hors-contexte

Exercice bonus 3. Bloquer les hallucinations polies

Pose une question dont la réponse **n'est pas** dans les pages indexées (par exemple : « *Comment installer un package avec pip ?* », sachant que la doc indexée couvre tutoriel + glossaire, pas l'installation). Observe le comportement par défaut. Puis durcis le prompt système pour que le modèle refuse explicitement de répondre hors-contexte.

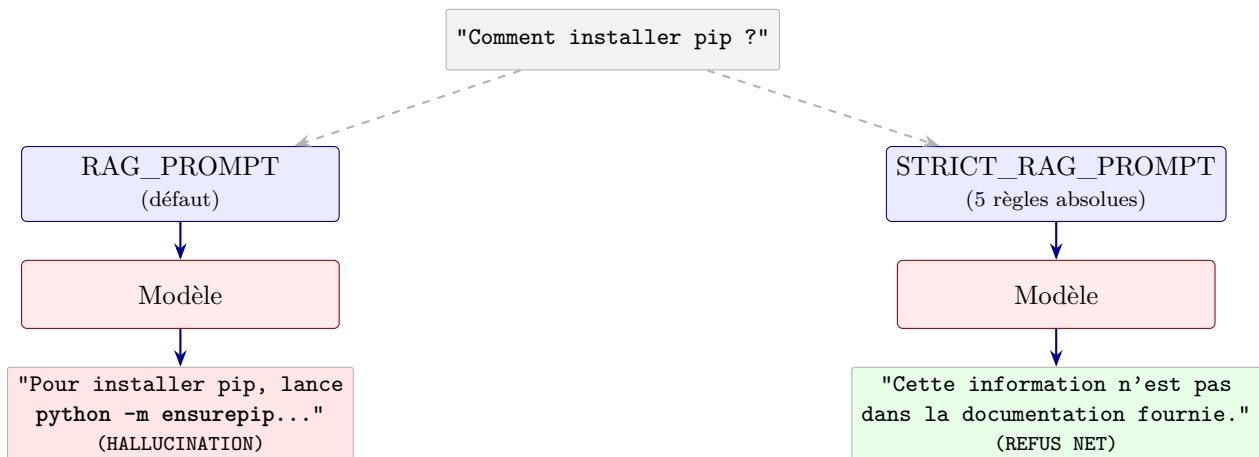
Contrainte : garder la même chaîne LCEL, juste swap le prompt. Le prompt strict doit **aussi** bien marcher sur des questions in-context (ne pas être excessivement restrictif).

Objectifs pédagogiques

- Reconnaître l'*hallucination polie* : le modèle complète avec ses connaissances générales quand le contexte ne suffit pas.
- Construire un prompt **défensif** avec règles numérotées et exemples de refus.
- Vérifier que le refus est exploitable en aval (router vers une autre source d'info, par exemple).

3.1 Solution

3.1.1 Architecture



3.1.2 Le code complet

```
1 from langchain_core.prompts import ChatPromptTemplate
2 from langchain_core.output_parsers import StrOutputParser
3 from langchain_core.runnables import RunnablePassthrough
4 from config import get_model
5 from phase2_rag import format_docs
6
7
8 STRICT_RAG_PROMPT = ChatPromptTemplate.from_messages([
9     (
10         "system",
11         "Tu réponds EXCLUSIVEMENT à partir du contexte fourni ci-dessous. "
12         "Règles absolues :\n"
13         "1. Si la réponse n'apparaît pas littéralement dans le contexte, "
14         "tu réponds : 'Cette information n'est pas dans la documentation fournie.' "
15         "et tu t'arrêtes.\n"
16         "2. Tu ne complètes JAMAIS avec tes connaissances générales sur Python.\n"
17         "3. Tu ne fais JAMAIS d'inférence en partant d'un sujet voisin.\n"
18         "4. Si tu utilises le contexte, cite les chunks par leur numéro [N].\n"
```

```

19         "5. Si le contexte est ambigu ou partiel, dis explicitement ce qui "
20         "manque plutôt que d'extrapoler."
21     ),
22     (
23         "human",
24         "Contexte :\n{context}\n\nQuestion : {question}",
25     ),
26 ])
27
28
29 def build_strict_rag_chain(retriever):
30     return (
31         {
32             "context": retriever | format_docs,
33             "question": RunnablePassthrough(),
34         }
35         | STRICT_RAG_PROMPT
36         | get_model()
37         | StrOutputParser()
38     )

```

3.1.3 Explications pas à pas

Étape 1 — L'hallucination polie

Sans contraintes explicites, un LLM moderne complète volontiers ses réponses avec ses connaissances générales : c'est ce qu'il a appris à faire pour être « utile ». Sur une question hors-corpus, il va :

- Ignorer (ou paraphraser) le contexte non pertinent.
- Sortir une réponse plausible et fluide à partir de ses *connaissances pré-entraînées*.
- Ne pas mentionner que cette réponse ne vient PAS du contexte.

C'est l'*hallucination polie* : la réponse est techniquement correcte, mais elle vient d'une source non-traçable. En contexte pédagogique, c'est exactement ce qu'on veut éviter.

Étape 2 — Le pattern *règles absolues numérotées*

```

1 "Règles absolues :
2 1. Si la réponse n'apparaît pas littéralement dans le contexte, ...
3 2. Tu ne complètes JAMAIS avec tes connaissances générales sur Python.
4 3. Tu ne fais JAMAIS d'inférence en partant d'un sujet voisin.
5 4. Si tu utilises le contexte, cite les chunks par leur numéro [N].
6 5. Si le contexte est ambigu ou partiel, dis explicitement ce qui manque."

```

Cinq caractéristiques d'un prompt défensif efficace :

- **Numérotation** : les modèles obéissent mieux à des règles listées qu'à un paragraphe.
- **Capitales** : EXCLUSIVEMENT, JAMAIS. Effet psychologique mesurable sur les LLM modernes.
- **Phrase exacte de refus** : « Cette information n'est pas dans la documentation fournie. » — le modèle a un *template* à recracher, pas à inventer.
- **Couverture explicite des cas voisins** : règle 3 anti- inférence (« cite-moi un sujet proche »), règle 5 anti-vague.
- **Garder la consigne positive** : règle 4 sur les citations, pour ne pas écraser le bon comportement par excès de restrictions.

Étape 3 — Vérifier la non-restriction excessive

Un piège classique des prompts stricts : ils deviennent *trop* restrictifs et refusent même des questions dont la réponse est dans le contexte. Test de contrôle indispensable :

```
1 in_context = "Qu'est-ce qu'un itérable en Python ?"      # dans le glossaire
2 print(chain_strict.invoke(in_context))                    # doit répondre normalement
```

Si le prompt strict refuse cette question, c'est qu'il est mal calibré — il faut adoucir le ton sans perdre la rigueur sur le hors-contexte.

Étape 4 — Le refus exploitable

L'intérêt d'un refus net (toujours la même phrase) plutôt que d'un brouillon hallucinant : le *code applicatif* peut le détecter et router :

```
1 answer = chain_strict.invoke(question)
2 if "Cette information n'est pas dans la documentation" in answer:
3     # router vers une recherche web, un autre corpus, etc.
4     fallback_chain.invoke(question)
```

*Un bon refus est **exploitable en aval**. L'hallucination polie ne l'est pas — il est impossible de savoir programmatiquement si la réponse est sourcée ou inventée. Toute architecture RAG sérieuse force le refus.*

3.2 Vérification dans LangSmith

Lance la même question hors-contexte avec les deux prompts. Dans LangSmith :

1. Compare les *outputs* côte-à-côte : l'un parle de pip, l'autre refuse.
2. Compare les *system prompts* : le strict a 4-5 fois plus de tokens (pour mémoire, ~200 tokens supplémentaires).
3. Compare les *tokens de sortie* : le refus est court (~30 tokens), l'hallucination est longue (~300 tokens). Le prompt strict réduit la facture sur les requêtes hors-corpus.

Récapitulatif

Ce qu'il faut retenir :

1. L'hallucination polie est le piège n°1 du RAG : le modèle complète sans le dire avec ses connaissances générales.
2. Prompt défensif efficace : règles **numérotées**, **capitales** sur les interdictions, phrase exacte de refus, couverture des cas voisins.
3. Toujours tester avec une question **in-context** aussi, pour ne pas créer un prompt trop restrictif.
4. Un refus net (phrase canonique) est **exploitable en aval** : on peut router vers une autre source.
5. Coût : prompt système ~200 tokens en plus, mais réponse plus courte sur hors-contexte → facture similaire ou inférieure.

4 Vérifier et exploiter LangSmith

Exercice bonus 4. Activer le tracing et lire l'arbre d'exécution

Configure LangSmith dans ton `.env`, exécute une chaîne RAG, et explore l'arbre d'exécution dans l'UI `smith.langchain.com`. Vérifie programmatiquement la présence des variables d'environnement nécessaires.

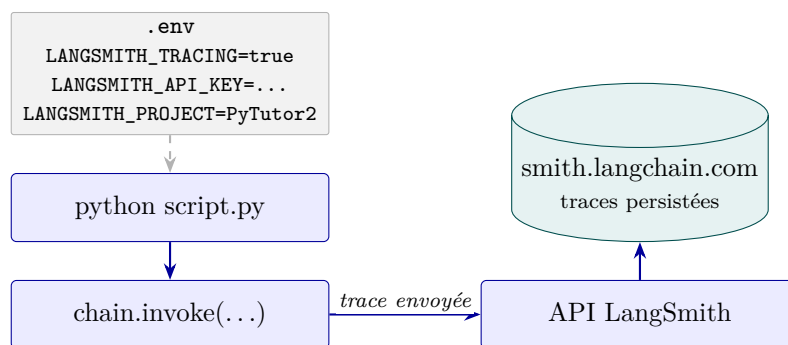
Contrainte : l'exercice doit afficher un message d'aide clair si la config est manquante (variables `LANGSMITH_TRACING`, `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT`), avec les lignes à ajouter au `.env`.

Objectifs pédagogiques

- Configurer LangSmith : variables d'environnement et endpoint EU vs US.
- Lire l'arbre d'exécution d'une chaîne RAG : les branches du `RunnableParallel`, le `format_docs`, le prompt rendu, l'appel modèle, le parser.
- Apprendre à **utiliser LangSmith comme principale source de vérité** sur le comportement d'une chaîne (pas le code, pas les logs).

4.1 Solution

4.1.1 Architecture



4.1.2 Le code complet

```
1 import os
2 from phase2_rag import build_or_load_vectorstore, build_rag_chain_with_sources
3
4
5 def exercise_4_langsmith() -> None:
6     # --- Vérification de la configuration ---
7     tracing = os.getenv("LANGSMITH_TRACING", "").lower() in {"true", "1"}
8     api_key = os.getenv("LANGSMITH_API_KEY")
9     project = os.getenv("LANGSMITH_PROJECT", "default")
10
11     print(f"LANGSMITH_TRACING : {'V activé' if tracing else 'X NON activé'}")
12     print(f"LANGSMITH_API_KEY : {'V présente' if api_key else 'X absente'}")
13     print(f"LANGSMITH_PROJECT : {project}")
14
15     if not (tracing and api_key):
16         print("→ Ajoute dans .env :")
17         print("    LANGSMITH_TRACING=true")
18         print("    LANGSMITH_API_KEY=lsv2_pt_...")
19         print("    LANGSMITH_PROJECT=PyTutor2")
20         print("    LANGSMITH_ENDPOINT=https://eu.api.smith.langchain.com # si compte  
    → EU")
```

```

21     return
22
23     # --- Exécution d'une chaîne pour générer une trace ---
24     vectorstore = build_or_load_vectorstore()
25     retriever    = vectorstore.as_retriever(search_kwargs={"k": 4})
26     chain        = build_rag_chain_with_sources(retriever)
27
28     result = chain.invoke("Comment écrire un générateur infini en Python ?")
29     print(result["answer"][:300] + "...")

```

4.1.3 Explications pas à pas

Étape 1 — Les 3 variables d'environnement essentielles

Variable	Rôle
LANGSMITH_TRACING	"true" pour activer l'envoi automatique des traces. Sans ça, les chaînes s'exécutent normalement mais rien n'est envoyé.
LANGSMITH_API_KEY	Clé personnelle (commence par lsv2_pt...). À créer sur smith.langchain.com (gratuit).
LANGSMITH_PROJECT	Nom du projet où les traces seront groupées. Ex. : PyTutor2.
LANGSMITH_ENDPOINT	Optionnel. https://eu.api.smith.langchain.com pour un compte EU (défaut : US).

Étape 2 — Vérification programmatique

Lire `os.getenv` avec un défaut explicite, et tester :

```

1 tracing = os.getenv("LANGSMITH_TRACING", "").lower() in {"true", "1"}
2 api_key = os.getenv("LANGSMITH_API_KEY")

```

Avantage : l'utilisateur voit *exactement* ce qui manque et a les lignes à coller dans son `.env`.

Étape 3 — L'arbre dans l'UI

Sur smith.langchain.com, ouvre ton projet et clique sur ta trace la plus récente. Le panneau gauche montre l'arbre d'exécution avec, pour la chaîne RAG :

1. Le `RunnableParallel` (les 2 branches `context` et `question`).
2. Le `Retriever` (avec sa query exacte et la liste de docs).
3. Le `format_docs` (un `RunnableLambda`).
4. Le `ChatPromptTemplate` (avec le prompt rendu, texte complet envoyé au LLM).
5. L'appel modèle (avec tokens consommés, latence, coût \$).
6. Le `StrOutputParser`.

Étape 4 — Pourquoi LangSmith est plus utile que les logs Python

Logs Python (print, logging)	LangSmith
Manuel : tu choisis quoi logger.	Automatique : tracé pour chaque Runnable.
Texte brut, dispersé sur stdout/fichier.	UI structurée, recherchable, partageable.
Pas de versionnage ni d'historique.	Toutes les traces archivées avec timestamp.
Pas de coût/latence agrégés.	Onglets dédiés <i>Costs</i> , <i>Latency</i> , <i>Tokens</i> .
Tu vois ce que TU as choisi de logger.	Tu vois exactement ce que le modèle a vu .

*LangSmith devient la principale source de vérité dès qu'on travaille sur une chaîne sérieuse. Les **print** restent utiles pour le local quick-and-dirty, mais la moindre investigation passe par l'UI.*

4.2 Vérification dans LangSmith

(Cet exercice EST la vérification.)

À regarder dans l'UI :

- L'onglet *Runs* liste toutes les traces récentes.
- Le panneau de *détail* d'une trace montre l'arbre.
- L'onglet *Costs* montre le coût \$ et les tokens.
- L'onglet *Annotations* permet de noter manuellement les bonnes/mauvaises réponses (*thumbs up/down*).

Récapitulatif

Ce qu'il faut retenir :

1. 3 variables d'env essentielles : `LANGSMITH_TRACING=true`, `LANGSMITH_API_KEY=lsu2_pt_...`, `LANGSMITH_PROJECT=ton-projet`.
2. Compte EU : ajouter `LANGSMITH_ENDPOINT=https://eu.api.smith.langchain.com`.
3. Vérification programmatique au démarrage de l'app : meilleure UX que de découvrir l'absence de trace plus tard.
4. L'UI LangSmith devient la principale source de vérité : arbre d'exécution, prompts rendus, coût, latence.
5. Annoter manuellement avec thumbs up/down crée un dataset d'évaluation gratuit.

5 MultiQueryRetriever : reformuler la question

Exercice bonus 5. Améliorer le recall avec un LLM en amont

Remplace le `Retriever` simple par un `MultiQueryRetriever` qui reformule chaque question utilisateur en 3 variantes via un LLM **avant** la recherche, puis fusionne les résultats des 3 recherches. Compare sur une question volontairement courte/ambiguë (ex. : « *Comment marche l'héritage ?* »).

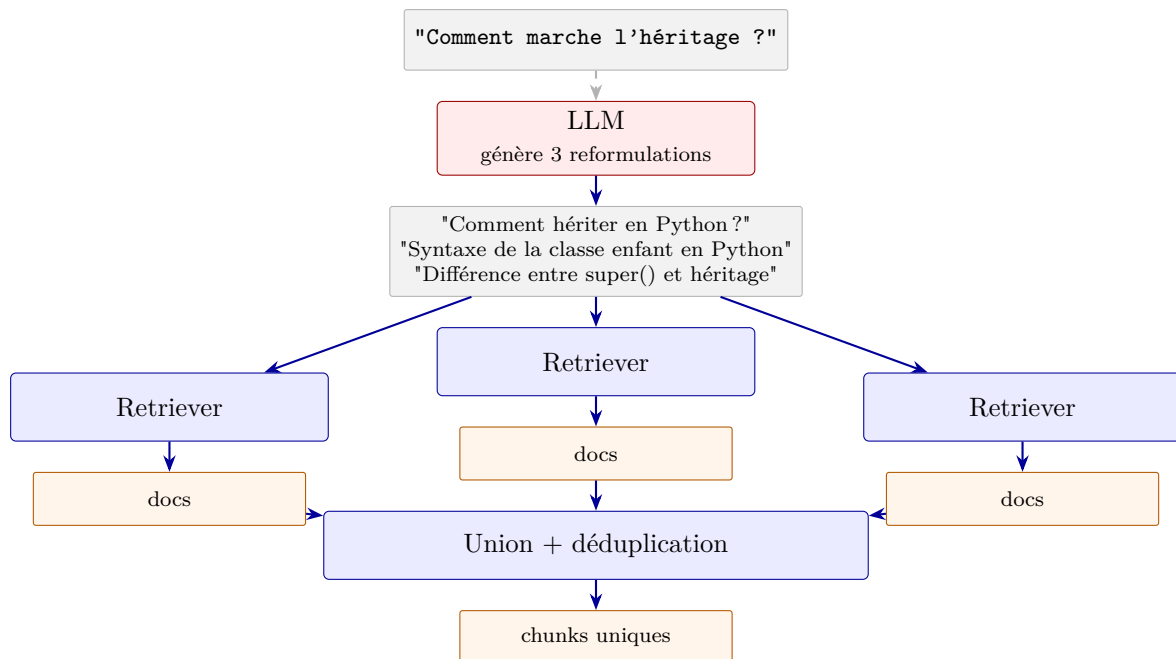
Contrainte : activer le logging du module `langchain.retrievers.multi_query` pour voir les reformulations générées par le LLM dans le terminal.

Objectifs pédagogiques

- Comprendre la notion de *recall* en information retrieval (retrouver toutes les bonnes infos, pas juste les plus pertinentes en surface).
- Utiliser un LLM pour **augmenter** la question utilisateur (et non pour répondre).
- Mesurer le coût supplémentaire (1 appel LLM de reformulation + N recherches au lieu d'une).
- Savoir quand utiliser MultiQuery : questions courtes / ambiguës ; et quand ne PAS l'utiliser : questions précises et bien formulées.

5.1 Solution

5.1.1 Architecture



Pipeline en quatre temps : reformulation LLM, N recherches parallèles, union, déduplication.

5.1.2 Le code complet

```
1 import logging
2 from langchain_classic.retrievers.multi_query import MultiQueryRetriever
3 from config import get_model
4 from phase2_rag import build_or_load_vectorstore, build_rag_chain
5
6
```

```

7 def exercise_5() -> None:
8     vectorstore = build_or_load_vectorstore()
9     base_retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
10    multi_retriever = MultiQueryRetriever.from_llm(
11        retriever=base_retriever,
12        llm=get_model(),
13    )
14
15    # Question courte et ambiguë : "héritage" peut couvrir plusieurs angles
16    question = "Comment marche l'héritage ?"
17
18    # --- Active le logging pour voir les reformulations ---
19    logging.basicConfig()
20    logging.getLogger("langchain.retrievers.multi_query").setLevel(logging.INFO)
21
22    # --- Retriever simple ---
23    docs_simple = base_retriever.invoke(question)
24    print(f"--- Simple (k=3) ---")
25    for i, d in enumerate(docs_simple, 1):
26        print(f" [{i}] {d.page_content[:100]}...")
27
28    # --- MultiQuery (les logs montrent les 3 reformulations) ---
29    docs_multi = multi_retriever.invoke(question)
30    print(f"\n--- MultiQuery ({len(docs_multi)} docs après déduplication) ---")
31    for i, d in enumerate(docs_multi, 1):
32        print(f" [{i}] {d.page_content[:100]}...")
33
34    # --- Comparaison des réponses finales ---
35    print("\n--- RÉPONSE simple ---")
36    print(build_rag_chain(base_retriever).invoke(question)[:400], "...")
37
38    print("\n--- RÉPONSE MultiQuery ---")
39    print(build_rag_chain(multi_retriever).invoke(question)[:400], "...")

```

5.1.3 Explications pas à pas

Étape 1 — *Precision vs Recall*

Métrique	Définition (en RAG)
Precision	Parmi les chunks retournés, combien sont pertinents ?
Recall	Parmi les chunks pertinents qui existent dans le corpus, combien sont retrouvés ?

Le retriever simple optimise la *precision* (top-k par similarité exacte). **MultiQuery** optimise le *recall* : en multipliant les formulations de la question, on **couvre plus d'angles** et on retrouve des chunks qui étaient invisibles à la formulation originale.

Étape 2 — Comment la reformulation marche

`MultiQueryRetriever.from_llm` envoie un prompt au LLM :

« Génère 3 reformulations différentes de la question suivante pour améliorer une recherche par similarité : "Comment marche l'héritage ?" »

Le LLM produit (par exemple) :

— « Comment hériter d'une classe en Python ? »

- « Quelle est la syntaxe d’une classe enfant qui hérite ? »
- « Différence entre héritage et composition en Python »

Chaque variante est ensuite passée au retriever de base. Les résultats sont **fusionnés et déduplicués** (par identité de chunk).

Étape 3 — Le logging interne

```
1 import logging
2 logging.basicConfig()
3 logging.getLogger("langchain.retrievers.multi_query").setLevel(logging.INFO)
```

Avec ces 3 lignes, les reformulations générées s’affichent dans la console à chaque appel. Indispensable pour :

- Comprendre ce que le LLM *reformule*.
- Diagnostiquer si les reformulations sont pertinentes ou triviales.
- Ajuster le **prompt** de reformulation au besoin.

Étape 4 — Coût et quand l’utiliser

Métrique	Simple	MultiQuery
Appels LLM pour la query	0 (embedding seul)	1 (reformulation)
Appels au retriever	1	N (typique : 3)
Coût par requête utilisateur	~1x	~1.5–2x
Gain sur questions précises	—	Nul , juste du coût
Gain sur questions courtes/ambiguës	—	Significatif (+30–50% de recall)

***Règle pratique** : utilise MultiQuery quand tu observes que des réponses sont incomplètes parce que des chunks pertinents n’ont pas été ramenés. N’utilise pas MultiQuery si le problème est ailleurs (chunk_size, prompt, modèle).*

5.2 Vérification dans LangSmith

Avec MultiQuery, LangSmith trace un arbre plus profond :

```
MultiQueryRetriever
├── LLMChain (génération des 3 reformulations) ← LLM #1
│   ├── VectorStoreRetriever (variante 1)
│   ├── VectorStoreRetriever (variante 2)
│   └── VectorStoreRetriever (variante 3)
│       └── (fusion + déduplication, hors trace)
```

*Tu vois directement les 3 variantes générées (onglet Output de la **LLMChain**), et le contenu des 3 retrievals. Précieux pour auditer la qualité des reformulations.*

Récapitulatif

Ce qu’il faut retenir :

1. **MultiQueryRetriever** reformule la question en *N* variantes **via un LLM**, puis fusionne les *N* résultats du retriever de base.

2. *Objectif* : améliorer le recall sur les questions courtes, ambiguës ou mal formulées.
3. *Coût* : +1 appel LLM pour reformuler + N appels au retriever. ~ 1.5 à $2\times$ le coût d'une requête simple.
4. Active le `logging` sur `langchain.retrievers.multi_query` pour voir les reformulations.
5. À ne PAS utiliser sur des questions déjà précises : aucun gain, juste du coût en plus.

6 Filtrage par métadonnée

Exercice bonus 6. Restreindre la recherche à une sous-partie du corpus

Chaque chunk du Chroma a hérité d'une `metadata.source` (l'URL d'origine). Construis un retriever filtré qui ne cherche que dans `tutorial/classes.html` pour une question sur la POO. Compare avec le retriever non filtré.

Démontre aussi le **piège** : si tu poses une question hors du périmètre du filtre (ex. : « Comment lever une exception ? ») alors que le filtre est sur `classes.html`, le retriever ramène du contenu hors-sujet.

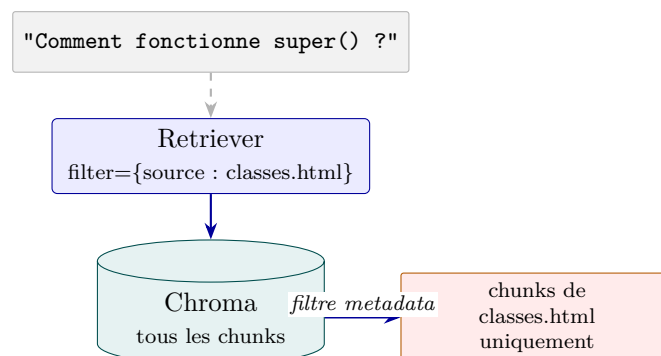
Contrainte : utiliser `search_kwargs={"filter": {...}}` avec la syntaxe MongoDB-like de Chroma.

Objectifs pédagogiques

- Utiliser la `metadata` des chunks pour restreindre la recherche.
- Connaître la syntaxe de filtre Chroma (`$eq`, `$in`, `$and`, `$or`, ...).
- Identifier le piège : un filtre mal calibré force le retriever à choisir le « moins pire » dans un sous-corpus inadapté.
- Esquisser le *pattern de production* : un classifieur LLM en amont qui décide *dynamiquement* du filtre à appliquer.

6.1 Solution

6.1.1 Architecture



Le filtre s'applique *avant* le calcul de similarité : seuls les chunks dont les metadata matchent sont éligibles à la recherche.

6.1.2 Le code complet

```

1 from phase2_rag import build_or_load_vectorstore
2
3
4 def exercice_6() -> None:
5     vectorstore = build_or_load_vectorstore()

```

```

6
7 # --- Retriever filtré : SEULEMENT classes.html ---
8 classes_only = vectorstore.as_retriever(
9     search_kwargs={
10         "k": 4,
11         "filter": {"source": "https://docs.python.org/3/tutorial/classes.html"},
12     },
13 )
14
15 # --- Retriever non filtré pour comparaison ---
16 unfiltered = vectorstore.as_retriever(search_kwargs={"k": 4})
17
18 question = "Comment fonctionne `super()` ?"
19
20 print("--- SANS filtre ---")
21 for i, d in enumerate(unfiltered.invoke(question), 1):
22     src = d.metadata.get("source", "?").split("/")[-1]
23     print(f" [{i}] ({src}) {d.page_content[:80]}...")
24
25 print("\n--- AVEC filtre source=classes.html ---")
26 for i, d in enumerate(classes_only.invoke(question), 1):
27     src = d.metadata.get("source", "?").split("/")[-1]
28     print(f" [{i}] ({src}) {d.page_content[:80]}...")
29
30 # Démo de la limite : question hors du périmètre du filtre
31 off_topic = "Comment lever une exception ?"
32 print(f"\n--- AVEC filtre, question hors-périmètre ({off_topic}) ---")
33 for i, d in enumerate(classes_only.invoke(off_topic), 1):
34     src = d.metadata.get("source", "?").split("/")[-1]
35     print(f" [{i}] ({src}) {d.page_content[:80]}...")

```

6.1.3 Explications pas à pas

Étape 1 — La syntaxe Chroma

```

1 search_kwargs={
2     "k": 4,
3     "filter": {"source": "https://docs.python.org/3/tutorial/classes.html"},
4 }

```

Forme simple : {key: value} = match exact sur la metadata. Sous le capot, Chroma traduit en {key: {\$eq: value}}.

Pour des filtres plus riches, syntaxe MongoDB-like :

Opérateur	Exemple
\$eq, \$ne	{"source": {"\$eq": "url"}}
\$in, \$nin	{"source": {"\$in": [url1, url2]}}
\$gt, \$gte, \$lt, \$lte	sur des champs numériques
\$and, \$or	{"\$and": [filtre1, filtre2]}

Étape 2 — Quand le filtre brille

Cas d'usage parfaits :

— **Multi-tenant** : chaque utilisateur a son propre corpus, filtrer par `tenant_id`.

- **Multi-langue** : filtrer par `lang` pour éviter de retrouver du contenu en anglais sur une query française.
- **Documentation versionnée** : filtrer par `version` (« `super()` en Python 3.0 vs 3.12 »).
- **Dates** : récupérer uniquement les chunks plus récents que X.

Étape 3 — Quand le filtre casse

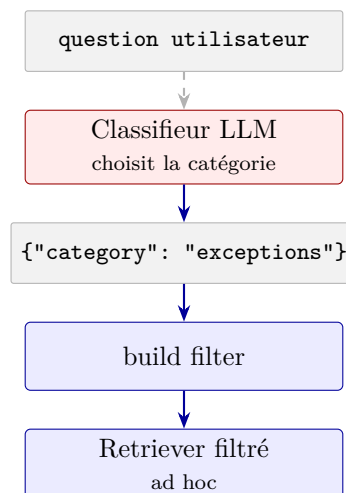
Si le filtre est **mal calibré** (utilisateur cherche dans le mauvais sous-corpus), le retriever ramène du contenu hors-sujet sans warning :

Sur « Comment lever une exception ? » avec `filter classes.html`, Chroma retourne les chunks de `classes.html` les plus proches de la query — qui parlent de classes, pas d'exceptions. Le RAG aval va générer une réponse hors-sujet ou hallucinée à partir de ce contexte inadapté.

C'est le danger du filtre : il **force** le retrieval à choisir dans un sous-ensemble, même si rien n'y est pertinent.

Étape 4 — Pattern de production : auto-routing

En production, le filtre ne devrait pas être codé en dur. Pattern typique :



Le mini-classifieur LLM (avec `with_structured_output + Literal[...]` comme dans la Phase 1 / Exercice 4 bonus) décide *dynamiquement* du filtre à appliquer. C'est l'analogue du `RunnableBranch` mais appliqué au retrieval.

6.2 Vérification dans LangSmith

Le run du retriever filtré expose le `filter` dans son input :

```

VectorStoreRetriever (filtered)
Input : "Comment fonctionne super() ?"
Config : k=4, filter={source: "../classes.html"}
Output : [Doc, Doc, Doc, Doc]
tous source: classes.html
  
```

Vérifie toujours dans LangSmith que la `metadata.source` des chunks retournés est bien dans le périmètre attendu. Si tu vois des sources hors du filtre, c'est qu'il y a un bug dans la spécification (souvent une typo dans l'URL).

Récapitulatif

Ce qu'il faut retenir :

1. *`search_kwargs={"filter": {...}}` restreint le retrieval aux chunks dont la **metadata** matche.*
2. *Syntaxe Chroma MongoDB-like : `$eq`, `$in`, `$and`, etc. Match exact si on omet l'opérateur.*
3. *Bons cas d'usage : multi-tenant, multi-langue, versionnage, dates.*
4. *Piège : si la question sort du périmètre du filtre, le retriever ramène du contenu hors-sujet sans warning. Toujours vérifier.*
5. *Pattern de production : classifieur LLM en amont qui décide dynamiquement du filtre à appliquer (auto-routing par metadata).*